



DEITEL® DEVELOPER SERIES



Python®

for Programmers

with introductory
AI case studies

- ▶ Natural Language Processing
- ▶ Data Mining Twitter®
- ▶ IBM® Watson™
- ▶ Machine Learning with scikit-learn®
- ▶ Deep Learning with Keras
- ▶ Big Data with Hadoop®, Spark™, NoSQL and the Cloud
- ▶ Internet of Things (IoT)
- ▶ Python Standard Library
- ▶ Data Science Libraries:
NumPy, Pandas, SciPy,
NLTK, TextBlob, Tweepy,
Matplotlib, Seaborn,
Folium and more

PAUL DEITEL • HARVEY DEITEL

Python[®] for Programmers

Deitel® Developer Series

Python for Programmers

Paul Deitel

Harvey Deitel





Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The authors and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the U.S., please contact intlcs@pearson.com.

Visit us on the Web: informit.com

Library of Congress Control Number: 2019933267

Copyright © 2019 Pearson Education, Inc.

All rights reserved. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, request forms, and the appropriate contacts within the Pearson Education Global Rights & Permissions Department, please visit www.pearsoned.com/permissions/.

Deitel and the double-thumbs-up bug are registered trademarks of Deitel and Associates, Inc.

Python logo courtesy of the Python Software Foundation.

Cover design by Paul Deitel, Harvey Deitel, and Chuti Prasertsith

Cover art by Afsandrew/Shutterstock

ISBN-13: 978-0-13-522433-5

ISBN-10: 0-13-522433-0

reface

“There’s gold in them thar hills!”¹

¹ Source unknown, frequently misattributed to Mark Twain.

Welcome to *Python for Programmers*! In this book, you’ll learn hands-on with today’s most compelling, leading-edge computing technologies, and you’ll program in Python—one of the world’s most popular languages and the fastest growing among them.

Developers often quickly discover that they like Python. They appreciate its expressive power, readability, conciseness and interactivity. They like the world of open-source software development that’s generating a rapidly growing base of reusable software for an enormous range of application areas.

For many decades, some powerful trends have been in place. Computer hardware has rapidly been getting faster, cheaper and smaller. Internet bandwidth has rapidly been getting larger and cheaper. And quality computer software has become ever more abundant and essentially free or nearly free through the “open source” movement. Soon, the “Internet of Things” will connect tens of billions of devices of every imaginable type. These will generate enormous volumes of data at rapidly increasing speeds and quantities.

In computing today, the latest innovations are “all about the data”—*data science*, *data analytics*, big *data*, relational *databases* (SQL), and NoSQL and NewSQL *databases*, each of which we address along with an innovative treatment of Python programming.

JOBS REQUIRING DATA SCIENCE SKILLS

In 2011, McKinsey Global Institute produced their report, “Big data: The next frontier for innovation, competition and productivity.” In it, they said, “The United States alone faces a shortage of 140,000 to 190,000 people with deep analytical skills as well as 1.5 million managers and analysts to analyze big data and make decisions based on their findings.”² This continues to be the case. The August 2018 “LinkedIn Workforce Report” says the United States has a shortage of over 150,000 people with data science skills.³ A 2017 report from IBM, Burning Glass Technologies and the Business-Higher Education Forum, says that by 2020 in the United States there will be hundreds of thousands of new jobs requiring data science skills.⁴

²

<https://www.mckinsey.com/~/media/McKinsey/Business%20Functions/McKinsey%20Digital/Our%20I>
page 3).

³ <https://economicgraph.linkedin.com/resources/linkedin-workforce-report-august-2018>.

⁴ https://www.burning-glass.com/wp-content/uploads/The_Quant_Crunch.pdf (page 3).

MODULAR ARCHITECTURE

The book’s **modular architecture** (please see the **Table of Contents graphic** on the book’s inside front cover) helps us meet the diverse needs of various professional audiences.

Chapters 1–10 cover Python programming. These chapters each include a brief **Intro to Data Science** section introducing artificial intelligence, basic descriptive statistics, measures of central tendency and dispersion, simulation, static and dynamic visualization, working with CSV files, pandas for data exploration and data wrangling, time series and

imple linear regression. These help you prepare for the data science, AI, big data and cloud case studies in [hapters 11– 6](#), which present opportunities for you to use **real-world datasets** in complete case studies.

After covering Python [hapters 1– 5](#) and a few key parts of [hapters 6– 7](#), you'll be able to handle significant portions of the case studies in [hapters 11– 6](#). The “Chapter Dependencies” section of this Preface will help trainers plan their professional courses in the context of the book's unique architecture.

[hapters 11– 6](#) are loaded with cool, powerful, contemporary examples. They present hands-on implementation case studies on topics such as **natural language processing**, **data mining Twitter**, **cognitive computing with IBM's Watson**, **supervised machine learning with classification and regression**, **unsupervised machine learning with clustering**, **deep learning with convolutional neural networks**, **deep learning with recurrent neural networks**, **big data with Hadoop, Spark and NoSQL databases**, the **Internet of Things** and more. Along the way, you'll acquire a **broad literacy** of data science terms and concepts, ranging from brief definitions to using concepts in small, medium and large programs. Browsing the book's detailed **Table of Contents** and Index will give you a sense of the breadth of coverage.

KEY FEATURES

KIS (Keep It Simple), KIS (Keep it Small), KIT (Keep it Topical)

- **Keep it simple**—In every aspect of the book, we strive for **simplicity and clarity**. For example, when we present natural language processing, we use the simple and intuitive **TextBlob** library rather than the more complex NLTK. In our deep learning presentation, we prefer **Keras** to TensorFlow. In general, when multiple libraries could be used to perform similar tasks, we use the simplest one.
- **Keep it small**—Most of the book's 538 examples are small—often just a few lines of code, with immediate interactive **IPython** feedback. We also include 40 larger scripts and in-depth case studies.
- **Keep it topical**—We read scores of recent Python-programming and data science books, and browsed, read or watched about 15,000 current articles, research papers, white papers, videos, blog posts, forum posts and documentation pieces. This enabled us to “take the pulse” of the Python, computer science, data science, AI, big data and cloud communities.

Immediate-Feedback: Exploring, Discovering and Experimenting with IPython

- The ideal way to learn from this book is to read it and run the code examples in parallel. Throughout the book, we use the **IPython interpreter**, which provides a friendly, immediate-feedback interactive mode for quickly exploring, discovering and experimenting with Python and its extensive libraries.
- Most of the code is presented in **small, interactive IPython sessions**. For each code snippet you write, IPython immediately reads it, evaluates it and prints the results. This **instant feedback** keeps your attention, boosts learning, facilitates rapid prototyping and speeds the software-development process.
- Our books always emphasize the **live-code approach**, focusing on complete, working programs with live inputs and outputs. IPython's “magic” is that it turns even snippets into code that “comes alive” as you enter each line. This promotes learning and encourages experimentation.

Python Programming Fundamentals

- First and foremost, this book provides rich Python coverage.
- We discuss Python's programming models—**procedural programming**, **functional-**

type programming and object-oriented programming.

- We use best practices, emphasizing current idiom.
- **Functional-style programming** is used throughout the book as appropriate. A chart in chapter 4 lists most of Python’s key functional-style programming capabilities and the chapters in which we initially cover most of them.

538 Code Examples

- You’ll get an engaging, challenging and entertaining introduction to Python with **538 real-world examples** ranging from individual snippets to substantial computer science, data science, artificial intelligence and big data case studies.
- You’ll attack significant tasks with **AI, big data and cloud** technologies like **natural language processing, data mining Twitter, machine learning, deep learning, Hadoop, MapReduce-, Spark, IBM Watson**, key data science libraries (**NumPy, pandas, SciPy, NLTK, TextBlob, spaCy, Textatistic, Tweepy, Scikit-learn, Keras**), key visualization libraries (**Matplotlib, Seaborn, Folium**) and more.

Avoid Heavy Math in Favor of English Explanations

- We capture the conceptual essence of the mathematics and put it to work in our examples. We do this by using libraries such as **statistics, NumPy, SciPy, pandas** and many others, which hide the mathematical complexity. So, it’s straightforward for you to get many of the benefits of mathematical techniques like **linear regression** without having to know the mathematics behind them. In the **machine-learning** and **deep-learning** examples, we focus on creating objects that do the math for you “behind the scenes.”

Visualizations

- 67 **static, dynamic, animated and interactive visualizations** (charts, graphs, pictures, animations etc.) help you understand concepts.
- Rather than including a treatment of low-level graphics programming, we focus on high-level visualizations produced by **Matplotlib, Seaborn, pandas** and **Folium** (for **interactive maps**).
- We use visualizations as a pedagogic tool. For example, we make the **law of large numbers** “come alive” in a dynamic **die-rolling simulation** and bar chart. As the number of rolls increases, you’ll see each face’s percentage of the total rolls gradually approach 16.667% (1/6th) and the sizes of the bars representing the percentages equalize.
- Visualizations are crucial in big data for **data exploration** and **communicating reproducible research results**, where the data items can number in the millions, billions or more. A common saying is that a picture is worth a thousand words ⁵ —in **big data**, a visualization could be worth billions, trillions or even more items in a database. Visualizations enable you to “fly 40,000 feet above the data” to see it “in the large” and to get to know your data. **Descriptive statistics** help but can be misleading. For example, Anscombe’s quartet ⁶ demonstrates through visualizations that *significantly different* datasets can have *nearly identical* descriptive statistics.

⁵

https://en.wikipedia.org/wiki/A_picture_is_worth_a_thousand_words.

⁶

https://en.wikipedia.org/wiki/Anscombe%27s_quartet.

- We show the visualization and animation code so you can implement your own. We also provide the animations in source-code files and as **Jupyter Notebooks**, so you can conveniently customize the code and animation parameters, re-execute the animations and see the effects of the changes.

Data Experiences

- Our **Intro to Data Science sections** and case studies in chapters 11– 16 provide rich data experiences.
- You'll work with many **real-world datasets and data sources**. There's an enormous variety of **free open datasets** available online for you to experiment with. Some of the sites we reference list hundreds or thousands of datasets.
- Many libraries you'll use come bundled with popular datasets for experimentation.
- You'll learn the steps required to obtain data and prepare it for analysis, analyze that data using many techniques, tune your models and communicate your results effectively, especially through visualization.

GitHub

- **GitHub** is an excellent venue for finding open-source code to incorporate into your projects (and to contribute your code to the open-source community). It's also a crucial element of the software developer's arsenal with version control tools that help teams of developers manage open-source (and private) projects.
- You'll use an extraordinary range of free and open-source Python and data science **libraries**, and **free**, **free-trial** and **freemium** offerings of software and cloud services. Many of the libraries are hosted on GitHub.

Hands-On Cloud Computing

- Much of big data analytics occurs in the cloud, where it's easy to scale *dynamically* the amount of hardware and software your applications need. You'll work with various cloud-based services (some directly and some indirectly), including **Twitter**, **Google Translate**, **IBM Watson**, **Microsoft Azure**, **OpenMapQuest**, **geopy**, **Dweet.io** and **PubNub**.
- We encourage you to use free, free trial or freemium cloud services. We prefer those that don't require a credit card because you don't want to risk accidentally running up big bills. **If you decide to use a service that requires a credit card, ensure that the tier you're using for free will not automatically jump to a paid tier.**

Database, Big Data and Big Data Infrastructure

- According to IBM (Nov. 2016), 90% of the world's data was created in the last two years.⁷ Evidence indicates that the speed of data creation is rapidly accelerating.

⁷

<https://public.dhe.ibm.com/common/ssi/ecm/wr/en/wr112345usen/watson-customer-engagement--watson-marketing-wr-other-papers-and-reports--r112345usen-20170719.pdf>.

- According to a March 2016 *AnalyticsWeek* article, within five years there will be over 50 billion devices connected to the Internet and by 2020 we'll be producing 1.7 megabytes of new data every second for *every person on the planet!*⁸

⁸

<https://analyticsweek.com/content/big-data-facts/>.

- We include a treatment of **relational databases** and **SQL** with **SQLite**.
- Databases are critical **big data infrastructure** for storing and manipulating the massive amounts of data you'll process. Relational databases process *structured data*—they're not geared to the *unstructured* and *semi-structured data* in big data applications. So, as big data evolved, **NoSQL and NewSQL databases** were created to handle such data efficiently. We include a NoSQL and NewSQL overview and a hands-on case study with a **MongoDB JSON document database**. MongoDB is the most popular NoSQL database.
- We discuss **big data hardware and software infrastructure** in chapter 16, “*Big Data Infrastructure*”.

ata: Hadoop, Spark, NoSQL and IoT (Internet of Things).”

Artificial Intelligence Case Studies

- In case study chapters 11–15, we present **artificial intelligence** topics, including **natural language processing**, **data mining** Twitter to perform **sentiment analysis**, **cognitive computing with IBM Watson**, **supervised machine learning**, **unsupervised machine learning** and **deep learning**. Chapter 16 presents the big data hardware and software infrastructure that enables computer scientists and data scientists to implement leading-edge AI-based solutions.

Built-In Collections: Lists, Tuples, Sets, Dictionaries

- There’s little reason today for most application developers to build *custom* data structures. The book features a rich **two-chapter treatment of Python’s built-in data structures—lists, tuples, dictionaries and sets**—with which most data-structuring tasks can be accomplished.

Array-Oriented Programming with NumPy Arrays and Pandas Series/DataFrames

- We also focus on three key data structures from open-source libraries—**NumPy arrays**, **pandas Series** and **pandas DataFrames**. These are used extensively in data science, computer science, artificial intelligence and big data. NumPy offers as much as two orders of magnitude higher performance than built-in Python lists.
- We include in chapter 7 a rich treatment of NumPy arrays. Many libraries, such as pandas, are built on NumPy. The **Intro to Data Science sections** in chapters 7–9 introduce pandas *Series* and *DataFrames*, which along with NumPy arrays are then used throughout the remaining chapters.

File Processing and Serialization

- Chapter 9 presents **text-file processing**, then demonstrates how to serialize objects using the popular **JSON (JavaScript Object Notation)** format. JSON is used frequently in the data science chapters.
- Many data science libraries provide built-in file-processing capabilities for loading datasets into your Python programs. In addition to plain text files, we process files in the popular **CSV (comma-separated values) format** using the Python Standard Library’s `csv` module and capabilities of the pandas data science library.

Object-Based Programming

- We emphasize using the huge number of valuable classes that the **Python open-source community** has packaged into industry standard class libraries. You’ll focus on knowing what libraries are out there, choosing the ones you’ll need for your apps, creating objects from existing classes (usually in one or two lines of code) and making them “jump, dance and sing.” This **object-based programming** enables you to build impressive applications quickly and concisely, which is a significant part of Python’s appeal.
- With this approach, you’ll be able to use machine learning, deep learning and other AI technologies to quickly solve a wide range of intriguing problems, including **cognitive computing** challenges like **speech recognition** and **computer vision**.

Object-Oriented Programming

- Developing *custom* classes is a crucial **object-oriented- programming** skill, along with inheritance, polymorphism and duck typing. We discuss these in chapter 10.
- Chapter 10 includes a discussion of **unit testing with doctest** and a fun card-shuffling-and-dealing simulation.

- [hapters 11– 6](#) require only a few straightforward custom class definitions. In Python, you'll probably use more of an object-based programming approach than full-out object-oriented programming.

Reproducibility

- In the sciences in general, and data science in particular, there's a need to reproduce the results of experiments and studies, and to communicate those results effectively. **Jupyter Notebooks** are a preferred means for doing this.
- We discuss reproducibility throughout the book in the context of programming techniques and software such as Jupyter Notebooks and **Docker**.

Performance

- We use the **%timeit profiling tool** in several examples to compare the performance of different approaches to performing the same tasks. Other performance-related discussions include generator expressions, NumPy arrays vs. Python lists, performance of machine-learning and deep-learning models, and Hadoop and Spark distributed-computing performance.

Big Data and Parallelism

- In this book, rather than writing your own parallelization code, you'll let libraries like Keras running over TensorFlow, and big data tools like Hadoop and Spark parallelize operations for you. In this big data/AI era, the sheer processing requirements of massive data applications demand taking advantage of true parallelism provided by **multicore processors, graphics processing units (GPUs), tensor processing units (TPUs)** and huge **clusters of computers in the cloud**. Some big data tasks could have thousands of processors working in parallel to analyze massive amounts of data expeditiously.

CHAPTER DEPENDENCIES

If you're a trainer planning your syllabus for a professional training course or a developer deciding which chapters to read, this section will help you make the best decisions. Please read the one-page color **Table of Contents** on the book's inside front cover—this will quickly familiarize you with the book's unique architecture. Teaching or reading the chapters in order is easiest. However, much of the content in the Intro to Data Science sections at the ends of [hapters 1– 0](#) and the case studies in [hapters 11– 6](#) requires only [hapters 1– 5](#) and small portions of [hapters 6– 0](#) as discussed below.

Part 1: Python Fundamentals Quickstart

We recommend that you read all the chapters in order:

- [hapter 1, Introduction to Computers and Python](#), introduces concepts that lay the groundwork for the Python programming in [hapters 2– 0](#) and the big data, artificial-intelligence and cloud-based case studies in [hapters 11– 6](#). The chapter also includes **test-drives** of the **IPython** interpreter and **Jupyter Notebooks**.
- [hapter 2, Introduction to Python Programming](#), presents Python programming fundamentals with code examples illustrating key language features.
- [hapter 3, Control Statements](#), presents Python's **control statements** and introduces **basic list processing**.
- [hapter 4, Functions](#), introduces custom functions, presents **simulation techniques** with **random-number generation** and introduces **tuple fundamentals**.
- [hapter 5, Sequences: Lists and Tuples](#), presents Python's built-in list and tuple collections in more detail and begins introducing **functional-style programming**.

Part 2: Python Data Structures, Strings and Files

The following summarizes inter-chapter dependencies for Python [chapters 6– 9](#) and assumes that you've read [chapters 1– 5](#).

- **chapter 6, Dictionaries and Sets**—The Intro to Data Science section in this chapter is not dependent on the chapter's contents.
- **chapter 7, Array-Oriented Programming with NumPy**—The Intro to Data Science section requires dictionaries ([chapter 6](#)) and arrays ([chapter 7](#)).
- **chapter 8, Strings: A Deeper Look**—The Intro to Data Science section requires raw strings and regular expressions ([sections 8.11– .12](#)), and pandas `Series` and `DataFrame` features from [section 7.14's](#) Intro to Data Science.
- **chapter 9, Files and Exceptions**—For **JSON serialization**, it's useful to understand dictionary fundamentals ([section 6.2](#)). Also, the Intro to Data Science section requires the built-in `open` function and the `with` statement ([section 9.3](#)), and pandas `DataFrame` features from [section 7.14's](#) Intro to Data Science.

Part 3: Python High-End Topics

The following summarizes inter-chapter dependencies for Python [chapter 10](#) and assumes that you've read [chapters 1– 5](#).

- **chapter 10, Object-Oriented Programming**—The Intro to Data Science section requires pandas `DataFrame` features from Intro to Data Science [section 7.14](#). Trainers wanting to cover only **classes and objects** can present [sections 10.1– 0.6](#). Trainers wanting to cover more advanced topics like **inheritance, polymorphism and duck typing**, can present [sections 10.7– 0.9](#). [Sections 10.10– 0.15](#) provide additional advanced perspectives.

Part 4: AI, Cloud and Big Data Case Studies

The following summary of inter-chapter dependencies for [chapters 11– 16](#) assumes that you've read [chapters 1– 5](#). Most of [chapters 11– 16](#) also require dictionary fundamentals from [section 6.2](#).

- **chapter 11, Natural Language Processing (NLP)**, uses pandas `DataFrame` features from [section 7.14's](#) Intro to Data Science.
- **chapter 12, Data Mining Twitter**, uses pandas `DataFrame` features from [section 7.14's](#) Intro to Data Science, string method `join` ([section 8.9](#)), JSON fundamentals ([section 9.5](#)), `TextBlob` ([section 11.2](#)) and Word clouds ([section 11.3](#)). Several examples require defining a class via inheritance ([chapter 10](#)).
- **chapter 13, IBM Watson and Cognitive Computing**, uses built-in function `open` and the `with` statement ([section 9.3](#)).
- **chapter 14, Machine Learning: Classification, Regression and Clustering**, uses NumPy array fundamentals and method `unique` ([chapter 7](#)), pandas `DataFrame` features from [section 7.14's](#) Intro to Data Science and Matplotlib function `subplots` ([section 10.6](#)).
- **chapter 15, Deep Learning**, requires NumPy array fundamentals ([chapter 7](#)), string method `join` ([section 8.9](#)), general machine-learning concepts from [chapter 14](#) and features from [chapter 14's](#) Case Study: Classification with k-Nearest Neighbors and the Digits Dataset.
- **chapter 16, Big Data: Hadoop, Spark, NoSQL and IoT**, uses string method `split` ([section 6.2.7](#)), Matplotlib `FuncAnimation` from [section 6.4's](#) Intro to Data Science, pandas `Series` and `DataFrame` features from [section 7.14's](#) Intro to Data Science, string

ethod `join` ([ection 8.9](#)), the `json` module ([ection 9.5](#)), NLTK stop words ([ection 1.2.13](#)) and from [chapter 12](#), Twitter authentication, Tweepy’s `StreamListener` class for streaming tweets, and the `geopy` and `folium` libraries. A few examples require defining a class via inheritance ([chapter 10](#)), but you can simply mimic the class definitions we provide without reading [chapter 10](#).

JUPYTER NOTEBOOKS

For your convenience, we provide the book’s code examples in **Python source code (.py) files** for use with the command-line IPython interpreter *and* as **Jupyter Notebooks (.ipynb) files** that you can load into your web browser and execute.

Jupyter Notebooks is a free, open-source project that enables you to combine text, graphics, audio, video, and interactive coding functionality for entering, editing, executing, debugging, and modifying code quickly and conveniently in a web browser. According to the article, “What Is Jupyter?”:

*Jupyter has become a standard for scientific research and data analysis. It packages computation and argument together, letting you build “computational narratives”; and it simplifies the problem of distributing working software to teammates and associates.*⁹

⁹ <https://www.oreilly.com/ideas/what-is-jupyter>.

In our experience, it’s a wonderful learning environment and **rapid prototyping tool**. For this reason, we use **Jupyter Notebooks** rather than a traditional IDE, such as **Eclipse**, **Visual Studio**, **PyCharm** or **Spyder**. Academics and professionals already use Jupyter extensively for sharing research results. Jupyter Notebooks support is provided through the traditional open-source community mechanisms⁹ (see “Getting Jupyter Help” later in this Preface). See the Before You Begin section that follows this Preface for software installation details and see the test-drives in [ection 1.5](#) for information on running the book’s examples.

⁹ <https://jupyter.org/community>.

Collaboration and Sharing Results

Working in teams and communicating research results are both important for developers in or moving into data-analytics positions in industry, government or academia:

- The notebooks you create are **easy to share** among team members simply by copying the files or via **GitHub**.
- Research results, including code and insights, can be shared as static web pages via tools like **nbviewer** (<https://nbviewer.jupyter.org>) and **GitHub**—both automatically render notebooks as web pages.

Reproducibility: A Strong Case for Jupyter Notebooks

In data science, and in the sciences in general, experiments and studies should be reproducible. This has been written about in the literature for many years, including

- Donald Knuth’s 1992 computer science publication—*Literate Programming*.¹

¹Knuth, D., “Literate Programming” (PDF), *The Computer Journal*, British Computer Society, 1992.

- The article “Language-Agnostic Reproducible Data Analysis Using Literate Programming,”² which says, “Lir (literate, reproducible computing) is based on the idea of literate programming as proposed by Donald Knuth.”

² <http://journals.plos.org/plosone/article?id=10.1371/journal.pone.0164023>.

Essentially, reproducibility captures the complete environment used to produce results—hardware, software, communications, algorithms (especially code), data and the **data’s**

rovenance (origin and lineage).

DOCKER

In [chapter 16](#), we'll use **Docker**—a tool for packaging software into containers that bundle everything required to execute that software conveniently, reproducibly and portably across platforms. Some software packages we use in [chapter 16](#) require complicated setup and configuration. For many of these, you can download free preexisting **Docker containers**. These enable you to avoid complex installation issues and execute software locally on your desktop or notebook computers, making Docker a great way to help you get started with new technologies quickly and conveniently.

Docker also helps with reproducibility. You can create custom Docker containers that are configured with the versions of every piece of software and every library you used in your study. This would enable other developers to recreate the environment you used, then reproduce your work, and will help you reproduce your own results. In [chapter 16](#), you'll use Docker to download and execute a container that's preconfigured for you to code and run big data Spark applications using Jupyter Notebooks.

SPECIAL FEATURE: IBM WATSON ANALYTICS AND COGNITIVE COMPUTING

Early in our research for this book, we recognized the rapidly growing interest in **IBM's Watson**. We investigated competitive services and found Watson's "no credit card required" policy for its "free tiers" to be among the most friendly for our readers.

IBM Watson is a **cognitive-computing** platform being employed across a wide range of real-world scenarios. Cognitive-computing systems simulate the **pattern-recognition** and **decision-making** capabilities of the human brain to "learn" as they consume more data.^{3,4,5} We include a significant hands-on Watson treatment. We use the free **Watson Developer Cloud: Python SDK**, which provides APIs that enable you to interact with Watson's services programmatically. Watson is fun to use and a great platform for letting your creative juices flow. You'll demo or use the following Watson APIs: **Conversation, Discovery, Language Translator, Natural Language Classifier, Natural Language Understanding, Personality Insights, Speech to Text, Text to Speech, Tone Analyzer** and **Visual Recognition**.

³ <http://whatis.techtarget.com/definition/cognitive-computing>.

⁴ https://en.wikipedia.org/wiki/Cognitive_computing.

⁵ <https://www.forbes.com/sites/bernardmarr/2016/03/23/what-everyone-should-know-about-cognitive-computing>.

Watson's Lite Tier Services and a Cool Watson Case Study

IBM encourages learning and experimentation by providing *free lite tiers* for many of its APIs.⁶ In [chapter 13](#), you'll try demos of many Watson services.⁷ Then, you'll use the lite tiers of Watson's **Text to Speech, Speech to Text** and **Translate** services to implement a **"traveler's assistant" translation app**. You'll speak a question in English, then the app will transcribe your speech to English text, translate the text to Spanish and speak the Spanish text. Next, you'll speak a Spanish response (in case you don't speak Spanish, we provide an audio file you can use). Then, the app will quickly transcribe the speech to Spanish text, translate the text to English and speak the English response. Cool stuff!

⁶ Always check the latest terms on IBM's website, as the terms and services may change.

⁷ <https://console.bluemix.net/catalog/>.

TEACHING APPROACH

Python for Programmers contains a rich collection of examples drawn from many fields. You'll work through interesting, real-world examples using **real-world datasets**. The book concentrates on the principles of good **software engineering** and stresses **program**

clarity.

Using Fonts for Emphasis

We place the key terms and the index's page reference for each defining occurrence in **bold** text for easier reference. We refer to on-screen components in the **bold Helvetica** font (for example, the **File** menu) and use the `Lucida` font for Python code (for example, `x = 5`).

Syntax Coloring

For readability, we syntax color all the code. Our syntax-coloring conventions are as follows:

```
comments appear in green
keywords appear in dark blue
constants and literal values appear in light blue
errors appear in red
all other code appears in black
```

538 Code Examples

The book's **538 examples** contain approximately **4000 lines of code**. This is a relatively small amount for a book this size and is due to the fact that Python is such an expressive language. Also, our coding style is to use powerful class libraries to do most of the work wherever possible.

160 Tables/Illustrations/Visualizations

We include abundant tables, line drawings, and static, dynamic and interactive visualizations.

Programming Wisdom

We *integrate* into the discussions programming wisdom from the authors' combined nine decades of programming and teaching experience, including:

- **Good programming practices** and preferred Python idioms that help you produce clearer, more understandable and more maintainable programs.
- **Common programming errors** to reduce the likelihood that you'll make them.
- **Error-prevention tips** with suggestions for exposing bugs and removing them from your programs. Many of these tips describe techniques for preventing bugs from getting into your programs in the first place.
- **Performance tips** that highlight opportunities to make your programs run faster or minimize the amount of memory they occupy.
- **Software engineering observations** that highlight architectural and design issues for proper software construction, especially for larger systems.

SOFTWARE USED IN THE BOOK

The software we use is available for Windows, macOS and Linux and is free for download from the Internet. We wrote the book's examples using the free **Anaconda Python distribution**. It includes most of the Python, visualization and data science libraries you'll need, as well as the IPython interpreter, Jupyter Notebooks and Spyder, considered one of the best Python data science IDEs. We use only IPython and Jupyter Notebooks for program development in the book. The Before You Begin section following this Preface discusses installing Anaconda and a few other items you'll need for working with our examples.

PYTHON DOCUMENTATION

You'll find the following documentation especially helpful as you work through the book:

- The Python Language Reference:

<https://docs.python.org/3/reference/index.html>

- The Python Standard Library:

<https://docs.python.org/3/library/index.html>

- Python documentation list:

<https://docs.python.org/3/>

GETTING YOUR QUESTIONS ANSWERED

Popular Python and general programming online forums include:

- python-forum.io
- <https://www.dreamincode.net/forums/forum/29-python/>
- [StackOverflow.com](https://stackoverflow.com)

Also, many vendors provide forums for their tools and libraries. Many of the libraries you'll use in this book are managed and maintained at github.com. Some library maintainers provide support through the **Issues** tab on a given library's GitHub page. If you cannot find an answer to your questions online, please see our web page for the book at

<http://www.deitel.com>⁸

⁸Our website is undergoing a major upgrade. If you do not find something you need, please write to us directly at deitel@deitel.com.

GETTING JUPYTER HELP

Jupyter Notebooks support is provided through:

- Project Jupyter Google Group:
<https://groups.google.com/forum/#!forum/jupyter>
- Jupyter real-time chat room:
<https://gitter.im/jupyter/jupyter>
- GitHub
<https://github.com/jupyter/help>
- StackOverflow:
<https://stackoverflow.com/questions/tagged/jupyter>
- Jupyter for Education Google Group (for instructors teaching with Jupyter):
<https://groups.google.com/forum/#!forum/jupyter-education>

SUPPLEMENTS

To get the most out of the presentation, you should execute each code example in parallel with reading the corresponding discussion in the book. On the book's web page at

<http://www.deitel.com>

we provide:

- **Downloadable Python source code** (.py files) and **Jupyter Notebooks** (.ipynb files) for the book's **code examples**.
- **Getting Started videos** showing how to use the code examples with IPython and Jupyter Notebooks. We also introduce these tools in [Section 1.5](#).

- **Blog posts** and **book updates**.

For download instructions, see the Before You Begin section that follows this Preface.

KEEPING IN TOUCH WITH THE AUTHORS

For answers to questions or to report an error, send an e-mail to us at

deitel@deitel.com

or interact with us via **social media**:

- **Facebook**[®] (<http://www.deitel.com/deitelfan>)
- **Twitter**[®] (@deitel)
- **LinkedIn**[®] (<http://linkedin.com/company/deitel-&-associates>)
- **YouTube**[®] (<http://youtube.com/DeitelTV>)

ACKNOWLEDGMENTS

We'd like to thank Barbara Deitel for long hours devoted to Internet research on this project. We're fortunate to have worked with the dedicated team of publishing professionals at Pearson. We appreciate the efforts and 25-year mentorship of our friend and colleague Mark L. Taub, Vice President of the Pearson IT Professional Group. Mark and his team publish our professional books, LiveLessons video products and Learning Paths in the Safari service (<https://learning.oreilly.com/>). They also sponsor our Safari live online training seminars. Julie Nahil managed the book's production. We selected the cover art and Chuti Prasertsith designed the cover.

We wish to acknowledge the efforts of our reviewers. Patricia Byron-Kimball and Meghan Jacoby recruited the reviewers and managed the review process. Adhering to a tight schedule, the reviewers scrutinized our work, providing countless suggestions for improving the accuracy, completeness and timeliness of the presentation.

Reviewers	
Book Reviewers	
Daniel Chen, Data Scientist, Lander Analytics	Daniel Chen, Data Scientist, Lander Analytics
Garrett Dancik, Associate Professor of Computer Science/Bioinformatics, Eastern Connecticut State University	Garrett Dancik, Associate Professor of Computer Science/Bioinformatics, Eastern Connecticut State University
Pranshu Gupta, Assistant Professor, Computer Science, DeSales University	Dr. Marsha Davis, Department Chair of Mathematical Sciences, Eastern Connecticut State University
David Koop, Assistant Professor, Data Science Program Co-Director, U-Mass Dartmouth	Roland DePratti, Adjunct Professor of Computer Science, Eastern Connecticut State University
Ramon Mata-Toledo, Professor, Computer Science, James Madison University	Shyamal Mitra, Senior Lecturer, Computer Science, University of Texas at Austin
Shyamal Mitra, Senior Lecturer, Computer Science, University of Texas at Austin	Dr. Mark Pauley, Senior Research Fellow, Bioinformatics, School of Interdisciplinary
Alison Sanchez, Assistant Professor in	

Economics, University of San Diego	Informatics, University of Nebraska at Omaha
José Antonio González Seco, IT Consultant	Sean Raleigh, Associate Professor of Mathematics, Chair of Data Science, Westminster College-
Jamie Whitacre, Independent Data Science Consultant	Alison Sanchez, Assistant Professor in Economics, University of San Diego
Elizabeth Wickes, Lecturer, School of Information Sciences, University of Illinois	Dr. Harvey Siy, Associate Professor of Computer Science, Information Science and Technology, University of Nebraska at Omaha
Proposal Reviewers	Jamie Whitacre, Independent Data Science Consultant
Dr. Irene Bruno, Associate Professor in the Department of Information Sciences and Technology, George Mason University	
Lance Bryant, Associate Professor, Department of Mathematics, Shippensburg University	

As you read the book, we'd appreciate your comments, criticisms, corrections and suggestions for improvement. Please send all correspondence to:

eitel@deitel.com

We'll respond promptly.

Welcome again to the exciting open-source world of Python programming. We hope you enjoy this look at leading-edge computer-applications development with Python, IPython, Jupyter Notebooks, data science, AI, big data and the cloud. We wish you great success!

Paul and Harvey Deitel

ABOUT THE AUTHORS

Paul J. Deitel, CEO and Chief Technical Officer of Deitel & Associates, Inc., is an MIT graduate with 38 years of experience in computing. Paul is one of the world's most experienced programming-languages trainers, having taught professional courses to software developers since 1992. He has delivered hundreds of programming courses to industry clients internationally, including Cisco, IBM, Siemens, Sun Microsystems (now Oracle), Dell, Fidelity, NASA at the Kennedy Space Center, the National Severe Storm Laboratory, White Sands Missile Range, Rogue Wave Software, Boeing, Nortel Networks, Puma, iRobot and many more. He and his co-author-, Dr. Harvey M. Deitel, are the world's best-selling programming-language textbook/professional book/video authors.

Dr. Harvey M. Deitel, Chairman and Chief Strategy Officer of Deitel & Associates, Inc., has 58 years of experience in computing. Dr. Deitel earned B.S. and M.S. degrees in Electrical Engineering from MIT and a Ph.D. in Mathematics from Boston University—he studied computing in each of these programs before they spun off Computer Science programs. He has extensive college teaching experience, including earning tenure and serving as the Chairman of the Computer Science Department at Boston College before founding Deitel & Associates, Inc., in 1991 with his son, Paul. The Deitels' publications have earned international recognition, with more than 100 translations published in Japanese, German, Russian, Spanish, French, Polish, Italian, Simplified Chinese, Traditional Chinese, Korean, Portuguese, Greek, Urdu and Turkish. Dr. Deitel has delivered hundreds of programming courses to academic, corporate, government and military clients.

ABOUT DEITEL® & ASSOCIATES, INC.

Deitel & Associates, Inc., founded by Paul Deitel and Harvey Deitel, is an internationally recognized authoring and corporate training organization, specializing in computer programming languages, object technology, mobile app development and Internet and web software technology. The company's training clients include some of the world's largest companies, government agencies, branches of the military and academic institutions. The company offers instructor-led training courses delivered at client sites worldwide on major programming languages.

Through its 44-year publishing partnership- with Pearson/Prentice Hall, Deitel & Associates, Inc., publishes leading-edge programming textbooks and professional books in print and e-book formats, **Live-Lessons** video courses (available for purchase at <https://www.informit.com>), **Learning Paths** and live online training seminars in the Safari service (<https://learning.oreilly.com>) and **Revel™** interactive multimedia courses.

To contact Deitel & Associates, Inc. and the authors, or to request a proposal on-site, instructor-led training, write to:

deitel@deitel.com

To learn more about Deitel on-site corporate training, visit

<http://www.deitel.com/training>

Individuals wishing to purchase Deitel books can do so at

<https://www.amazon.com>

Bulk orders by corporations, the government, the military and academic institutions should be placed directly with Pearson. For more information, visit

<https://www.informit.com/store/sales.aspx>

Before You Begin

This section contains information you should review before using this book. We'll post updates at: <http://www.deitel.com>.

FONT AND NAMING CONVENTIONS

We show Python code and commands and file and folder names in a `sans-serif` font, and on-screen components, such as menu names, in a **bold sans-serif font**. We use *italics for emphasis* and **bold occasionally for strong emphasis**.

GETTING THE CODE EXAMPLES

You can download the `examples.zip` file containing the book's examples from our *Python for Programmers* web page at:

<http://www.deitel.com>

Click the **Download Examples** link to save the file to your local computer. Most web browsers place the file in your user account's `Downloads` folder. When the download completes, locate it on your system, and extract its `examples` folder into your user account's `Documents` folder:

- **Windows:** `C:\Users\YourAccount\Documents\examples`
- **macOS or Linux:** `~/Documents/examples`

Most operating systems have a built-in extraction tool. You also may use an archive tool such as 7-Zip (www.7-zip.org) or WinZip (www.winzip.com).

STRUCTURE OF THE EXAMPLES FOLDER

You'll execute three kinds of examples in this book:

- Individual code snippets in the IPython interactive environment.
- Complete applications, which are known as scripts.
- Jupyter Notebooks—a convenient interactive, web-browser-based environment in which you can write and execute code and intermix the code with text, images and video.

We demonstrate each in [Section 1.5](#)'s test drives.

The `examples` folder contains one subfolder per chapter. These are named `ch##`, where `##` is the two-digit chapter number 01 to 16—for example, `ch01`. Except for [chapters 13, 15 and 16](#), each chapter's folder contains the following items:

- `snippets_ipynb`—A folder containing the chapter's Jupyter Notebook files.
- `snippets_py`—A folder containing Python source code files in which each code snippet we present is separated from the next by a blank line. You can copy and paste these snippets into IPython or into new Jupyter Notebooks that you create.
- Script files and their supporting files.

[Chapter 13](#) contains one application. [Chapters 15 and 16](#) explain where to find the files you need in the `ch15` and `ch16` folders, respectively.

INSTALLING ANACONDA

We use the easy-to-install Anaconda Python distribution with this book. It comes with almost everything you'll need to work with our examples, including:

- the IPython interpreter,
- most of the Python and data science libraries we use,
- a local Jupyter Notebooks server so you can load and execute our notebooks, and
- various other software packages, such as the Spyder Integrated Development Environment (IDE)—we use only IPython and Jupyter Notebooks in this book.

Download the Python 3.x Anaconda installer for Windows, macOS or Linux from:

<https://www.anaconda.com/download/>

When the download completes, run the installer and follow the on-screen instructions. To ensure that Anaconda runs correctly, do not move its files after you install it.

UPDATING ANACONDA

Next, ensure that Anaconda is up to date. Open a command-line window on your system as follows:

- On macOS, open a **Terminal** from the **Applications** folder's **Utilities** subfolder.
- On Windows, open the **Anaconda Prompt** from the start menu. When doing this to update Anaconda (as you'll do here) or to install new packages (discussed momentarily), execute the **Anaconda Prompt** as an *administrator* by right-clicking, then selecting **More > Run as administrator**. (If you cannot find the Anaconda Prompt in the start menu, simply search for it in the **Type here to search** field at the bottom of your screen.)
- On Linux, open your system's **Terminal** or shell (this varies by Linux distribution).

In your system's command-line window, execute the following commands to update Anaconda's installed packages to their latest versions:

1. `conda update conda`

2. `conda update --all`

PACKAGE MANAGERS

The `conda` command used above invokes the **conda package manager**—one of the two key Python package managers you'll use in this book. The other is **pip**. Packages contain the files required to install a given Python library or tool. Throughout the book, you'll use `conda` to install additional packages, unless those packages are not available through `conda`, in which case you'll use `pip`. Some people prefer to use `pip` exclusively as it currently supports more packages. If you ever have trouble installing a package with `conda`, try `pip` instead.

INSTALLING THE PROSPECTOR STATIC CODE ANALYSIS TOOL

ou may want to analyze your Python code using the Prospector analysis tool, which checks your code for common errors and helps you improve it. To install Prospector and the Python libraries it uses, run the following command in the command-line window:

```
pip install prospector
```

INSTALLING JUPYTER-MATPLOTLIB

We implement several animations using a visualization library called Matplotlib. To use them in Jupyter Notebooks, you must install a tool called `ipyml`. In the **Terminal**, **Anaconda Command Prompt** or shell you opened previously, execute the following commands ¹ one at a time:

¹ <https://github.com/matplotlib/jupyter-matplotlib>.

```
conda install -c conda-forge ipympl
conda install nodejs
jupyter labextension install @jupyter-widgets/jupyterlab-manager
jupyter labextension install jupyter-matplotlib
```

INSTALLING THE OTHER PACKAGES

Anaconda comes with approximately 300 popular Python and data science packages for you, such as NumPy, Matplotlib, pandas, Regex, BeautifulSoup, requests, Bokeh, SciPy, SciKit-Learn, Seaborn, Spacy, sqlite, statsmodels and many more. The number of additional packages you'll need to install throughout the book will be small and we'll provide installation instructions as necessary. As you discover new packages, their documentation will explain how to install them.

GET A TWITTER DEVELOPER ACCOUNT

If you intend to use our “Data Mining Twitter” chapter and any Twitter-based examples in subsequent chapters, apply for a Twitter developer account. Twitter now requires registration for access to their APIs. To apply, fill out and submit the application at

<https://developer.twitter.com/en/apply-for-access>

Twitter reviews every application. At the time of this writing, personal developer accounts were being approved immediately and company-account applications were

aking from several days to several weeks. Approval is not guaranteed.

INTERNET CONNECTION REQUIRED IN SOME CHAPTERS

While using this book, you'll need an Internet connection to install various additional Python libraries. In some chapters, you'll register for accounts with cloud-based services, mostly to use their free tiers. Some services require credit cards to verify your identity. In a few cases, you'll use services that are not free. In these cases, you'll take advantage of monetary credits provided by the vendors so you can try their services without incurring charges. **Caution: Some cloud-based services incur costs once you set them up. When you complete our case studies using such services, be sure to promptly delete the resources you allocated.**

SLIGHT DIFFERENCES IN PROGRAM OUTPUTS

When you execute our examples, you might notice some differences between the results we show and your own results:

- Due to differences in how calculations are performed with floating-point numbers (like `-123.45`, `7.5` or `0.0236937`) across operating systems, you might see minor variations in outputs—especially in digits far to the right of the decimal point.
- When we show outputs that appear in separate windows, we crop the windows to remove their borders.

GETTING YOUR QUESTIONS ANSWERED

Online forums enable you to interact with other Python programmers and get your Python questions answered. Popular Python and general programming forums include:

- `python-forum.io`
- `StackOverflow.com`
- `https://www.dreamincode.net/forums/forum/29-python/`

Also, many vendors provide forums for their tools and libraries. Most of the libraries you'll use in this book are managed and maintained at `github.com`. Some library maintainers provide support through the **Issues** tab on a given library's GitHub page.

f you cannot find an answer to your questions online, please see our web page for the book at

<http://www.deitel.com>²

² Our website is undergoing a major upgrade. If you do not find something you need, please write to us directly at deitel@deitel.com.

You're now ready to begin reading *Python for Programmers*. We hope you enjoy the book!

1. Introduction to Computers and Python

Objectives

In this chapter you'll:

- Learn about exciting recent developments in computing.
- Review object-oriented programming basics.
- Understand the strengths of Python.
- Be introduced to key Python and data-science libraries you'll use in this book.
- Test-drive the IPython interpreter's interactive mode for executing Python code.
- Execute a Python script that animates a bar chart.
- Create and test-drive a web-browser-based Jupyter Notebook for executing Python code.
- Learn how big “big data” is and how quickly it's getting even bigger.
- Read a big-data case study on a popular mobile navigation app.
- Be introduced to artificial intelligence—at the intersection of computer science and data science.

Outline

.1 Introduction

.2 A Quick Review of Object Technology Basics

.3 Python

.4 It's the Libraries!

.4.1 Python Standard Library

.4.2 Data-Science Libraries

.5 Test-Drives: Using IPython and Jupyter Notebooks

.5.1 Using IPython Interactive Mode as a Calculator

.5.2 Executing a Python Program Using the IPython Interpreter

.5.3 Writing and Executing Code in a Jupyter Notebook

.6 The Cloud and the Internet of Things

.6.1 The Cloud

.6.2 Internet of Things

.7 How Big Is Big Data?

.7.1 Big Data Analytics

.7.2 Data Science and Big Data Are Making a Difference: Use Cases

.8 Case Study—A Big-Data Mobile Application

.9 Intro to Data Science: Artificial Intelligence—at the Intersection of CS and Data Science

.10 Wrap-Up

1.1 INTRODUCTION

Welcome to Python—one of the world’s most widely used computer programming languages and, according to the *Popularity of Programming Languages (PYPL) Index*, the world’s most popular.¹

¹ <https://pypl.github.io/PYPL.html> (as of January 2019).

Here, we introduce terminology and concepts that lay the groundwork for the Python programming you’ll learn in **hapters 2– 0** and the big-data, artificial-intelligence and cloud-based case studies we present in **hapters 11– 6**.

We’ll review *object-oriented programming* terminology and concepts. You’ll learn why Python has become so popular. We’ll introduce the Python Standard Library and various data-science libraries that help you avoid “reinventing the wheel.” You’ll use these libraries to create software objects that you’ll interact with to perform significant tasks with modest numbers of instructions.

Next, you’ll work through three test-drives showing how to execute Python code:

- In the first, you'll use IPython to execute Python instructions interactively and immediately see their results.
- In the second, you'll execute a substantial Python application that will display an animated bar chart summarizing rolls of a six-sided die as they occur. You'll see the “*Law of Large Numbers*” in action. In *Chapter 6*, you'll build this application with the Matplotlib visualization library.
- In the last, we'll introduce Jupyter Notebooks using JupyterLab—an interactive, web-browser-based tool in which you can conveniently write and execute Python instructions. Jupyter Notebooks enable you to include text, images, audios, videos, animations and code.

In the past, most computer applications ran on standalone computers (that is, not networked together). Today's applications can be written with the aim of communicating among the world's billions of computers via the Internet. We'll introduce the Cloud and the Internet of Things (IoT), laying the groundwork for the contemporary applications you'll develop in *Chapters 11–16*.

You'll learn just how big “big data” is and how quickly it's getting even bigger. Next, we'll present a big-data case study on the Waze mobile navigation app, which uses many current technologies to provide dynamic driving directions that get you to your destination as quickly and as safely as possible. As we walk through those technologies, we'll mention where you'll use many of them in this book. The chapter closes with our first Intro to Data Science section in which we discuss a key intersection between computer science and data science—artificial intelligence.

1.2 A QUICK REVIEW OF OBJECT TECHNOLOGY BASICS

As demands for new and more powerful software are soaring, building software quickly, correctly and economically is important. *Objects*, or more precisely, the *classes* objects come from, are essentially *reusable* software components. There are date objects, time objects, audio objects, video objects, automobile objects, people objects, etc. Almost any *noun* can be reasonably represented as a software object in terms of *attributes* (e.g., name, color and size) and *behaviors* (e.g., calculating, moving and communicating). Software-development groups can use a modular, object-oriented design-and-implementation approach to be much more productive than with earlier popular techniques like “structured programming.” Object-oriented programs are often easier to understand, correct and modify.

Automobile as an Object

To help you understand objects and their contents, let's begin with a simple analogy. Suppose you want to *drive a car and make it go faster by pressing its accelerator pedal*. What must happen before you can do this? Well, before you can drive a car, someone has to *design* it. A car typically begins as engineering drawings, similar to the *blueprints* that describe the

esign of a house. These drawings include the design for an accelerator pedal. The pedal *hides* from the driver the complex mechanisms that make the car go faster, just as the brake pedal “hides” the mechanisms that slow the car, and the steering wheel “hides” the mechanisms that turn the car. This enables people with little or no knowledge of how engines, braking and steering mechanisms work to drive a car easily.

Just as you cannot cook meals in the blueprint of a kitchen, you cannot drive a car’s engineering drawings. Before you can drive a car, it must be *built* from the engineering drawings that describe it. A completed car has an *actual* accelerator pedal to make it go faster, but even that’s not enough—the car won’t accelerate on its own (hopefully!), so the driver must *press* the pedal to accelerate the car.

Methods and Classes

Let’s use our car example to introduce some key object-oriented programming concepts. Performing a task in a program requires a **method**. The method houses the program statements that perform its tasks. The method hides these statements from its user, just as the accelerator pedal of a car hides from the driver the mechanisms of making the car go faster. In Python, a program unit called a **class** houses the set of methods that perform the class’s tasks. For example, a class that represents a bank account might contain one method to *deposit* money to an account, another to *withdraw* money from an account and a third to *inquire* what the account’s balance is. A class is similar in concept to a car’s engineering drawings, which house the design of an accelerator pedal, steering wheel, and so on.

Instantiation

Just as someone has to *build a car* from its engineering drawings before you can drive a car, you must *build an object* of a class before a program can perform the tasks that the class’s methods define. The process of doing this is called *instantiation*. An object is then referred to as an **instance** of its class.

Reuse

Just as a car’s engineering drawings can be *reused* many times to build many cars, you can *reuse* a class many times to build many objects. Reuse of existing classes when building new classes and programs saves time and effort. Reuse also helps you build more reliable and effective systems because existing classes and components often have undergone extensive *testing*, *debugging* and *performance tuning*. Just as the notion of *interchangeable parts* was crucial to the Industrial Revolution, reusable classes are crucial to the software revolution that has been spurred by object technology.

In Python, you’ll typically use a *building-block approach* to create your programs. To avoid reinventing the wheel, you’ll use existing high-quality pieces wherever possible. This software reuse is a key benefit of object-oriented programming.

Messages and Method Calls

When you drive a car, pressing its gas pedal sends a *message* to the car to perform a task—that is, to go faster. Similarly, you *send messages to an object*. Each message is implemented as a method call that tells a method of the object to perform its task. For example, a program might call a bank-account object’s *deposit* method to increase the account’s balance.

Attributes and Instance Variables

A car, besides having capabilities to accomplish tasks, also has *attributes*, such as its color, its number of doors, the amount of gas in its tank, its current speed and its record of total miles driven (i.e., its odometer reading). Like its capabilities, the car’s attributes are represented as part of its design in its engineering diagrams (which, for example, include an odometer and a fuel gauge). As you drive an actual car, these attributes are carried along with the car. Every car maintains its *own* attributes. For example, each car knows how much gas is in its own gas tank, but *not* how much is in the tanks of *other* cars.

An object, similarly, has attributes that it carries along as it’s used in a program. These attributes are specified as part of the object’s class. For example, a bank-account object has a *balance attribute* that represents the amount of money in the account. Each bank-account object knows the balance in the account it represents, but *not* the balances of the *other* accounts in the bank. Attributes are specified by the class’s **instance variables**. A class’s (and its object’s) attributes and methods are intimately related, so classes wrap together their attributes and methods.

Inheritance

A new class of objects can be created conveniently by **inheritance**—the new class (called the **subclass**) starts with the characteristics of an existing class (called the **superclass**), possibly customizing them and adding unique characteristics of its own. In our car analogy, an object of class “convertible” certainly *is an* object of the more *general* class “automobile,” but more *specifically*, the roof can be raised or lowered.

Object-Oriented Analysis and Design (OOAD)

Soon you’ll be writing programs in Python. How will you create the code for your programs? Perhaps, like many programmers, you’ll simply turn on your computer and start typing. This approach may work for small programs (like the ones we present in the early chapters of the book), but what if you were asked to create a software system to control thousands of automated teller machines for a major bank? Or suppose you were asked to work on a team of 1,000 software developers building the next generation of the U.S. air traffic control system? For projects so large and complex, you should not simply sit down and start writing programs.

To create the best solutions, you should follow a detailed **analysis** process for determining your project’s **requirements** (i.e., defining *what* the system is supposed to do), then develop a **design** that satisfies them (i.e., specifying *how* the system should do it). Ideally, you’d go through this process and carefully review the design (and have your design reviewed

y other software professionals) before writing any code. If this process involves analyzing and designing your system from an object-oriented point of view, it's called an **object-oriented analysis-and-design (OOAD) process**. Languages like Python are object-oriented. Programming in such a language, called **object-oriented programming (OOP)**, allows you to implement an object-oriented design as a working system.

1.3 PYTHON

Python is an object-oriented scripting language that was released publicly in 1991. It was developed by Guido van Rossum of the National Research Institute for Mathematics and Computer Science in Amsterdam.

Python has rapidly become one of the world's most popular programming languages. It's now particularly popular for educational and scientific computing,² and it recently surpassed the programming language R as the most popular data-science programming language.^{3, 4, 5} Here are some reasons why Python is popular and everyone should consider learning it:^{6, 7, 8}

² <https://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017>.

³ <https://www.kdnuggets.com/2017/08/python-overtakes-r-leader-analytics-data-science.html>.

⁴ <https://www.r-bloggers.com/data-science-job-report-2017-r-passes-as-but-python-leaves-them-both-behind/>.

⁵ <https://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017>.

⁶ <https://dbader.org/blog/why-learn-python>.

⁷ <https://simpleprogrammer.com/2017/01/18/7-reasons-why-you-should-learn-python/>.

⁸ <https://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017>.

- It's open source, free and widely available with a massive open-source community.
- It's easier to learn than languages like C, C++, C# and Java, enabling novices and professional developers to get up to speed quickly.
- It's easier to read than many other popular programming languages.
- It's widely used in education.⁹

⁹ Tollervey, N., *Python in Education: Teach, Learn, Program* (O'Reilly Media, Inc., 2015).

- It enhances developer productivity with extensive standard libraries and third-party open-source libraries, so programmers can write code faster and perform complex tasks with minimal code. We'll say more about this in [Section 1.4](#).
- There are massive numbers of free open-source Python applications.
- It's popular in web development (e.g., Django, Flask).
- It supports popular programming paradigms—procedural, functional-style and object-oriented.⁰ We'll begin introducing functional-style programming features in [Chapter 4](#) and use them in subsequent chapters.

⁰ [https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language)).

- It simplifies concurrent programming—with `asyncio` and `async/await`, you're able to write single-threaded concurrent code¹, greatly simplifying the inherently complex processes of writing, debugging and maintaining that code.²

¹ <https://docs.python.org/3/library/asyncio.html>.

² <https://www.oreilly.com/ideas/5-things-to-watch-in-python-in-2017>.

- There are lots of capabilities for enhancing Python performance.
- It's used to build anything from simple scripts to complex apps with massive numbers of users, such as Dropbox, YouTube, Reddit, Instagram and Quora.³

³ https://www.hartmannsoftware.com/Blog/Articles_from_Software_Fans/Mostamous-Software-Programs-Written-in-Python.

- It's popular in artificial intelligence, which is enjoying explosive growth, in part because of its special relationship with data science.
- It's widely used in the financial community.⁴

⁴Kolanovic, M. and R. Krishnamachari, *Big Data and AI Strategies: Machine Learning and Alternative Data Approach to Investing* (J.P. Morgan, 2017).

- There's an extensive job market for Python programmers across many disciplines, especially in data-science--oriented positions, and Python jobs are among the highest paid of all programming jobs.^{5, 6}

⁵ <https://www.infoworld.com/article/3170838/developer/get-paid-10-programming-languages-to-learn-in-2017.html>.

⁶ <https://medium.com/@ChallengeRocket/top-10-of-programming->

- R is a popular open-source programming language for statistical applications and visualization. Python and R are the two most widely data-science languages.

Anaconda Python Distribution

We use the Anaconda Python distribution because it's easy to install on Windows, macOS and Linux and supports the latest versions of Python, the IPython interpreter (introduced in [section 1.5.1](#)) and Jupyter Notebooks (introduced in [section 1.5.3](#)). Anaconda also includes other software packages and libraries commonly used in Python programming and data science, allowing you to focus on Python and data science, rather than software installation issues. The IPython interpreter ⁷ has features that help you explore, discover and experiment with Python, the Python Standard Library and the extensive set of third-party libraries.

⁷ <https://ipython.org/>.

Zen of Python

We adhere to Tim Peters' *The Zen of Python*, which summarizes Python creator Guido van Rossum's design principles for the language. This list can be viewed in IPython with the command `import this`. The Zen of Python is defined in Python Enhancement Proposal (PEP) 20. "A PEP is a design document providing information to the Python community, or describing a new feature for Python or its processes or environment." ⁸

⁸ <https://www.python.org/dev/peps/pep-0001/>.

1.4 IT'S THE LIBRARIES!

Throughout the book, we focus on using existing libraries to help you avoid "reinventing the wheel," thus leveraging your program-development efforts. Often, rather than developing lots of original code—a costly and time-consuming process—you can simply create an object of a pre-existing library class, which takes only a single Python statement. So, libraries will help you perform significant tasks with modest amounts of code. In this book, you'll use a broad range of Python standard libraries, data-science libraries and third-party libraries.

1.4.1 Python Standard Library

The **Python Standard Library** provides rich capabilities for text/binary data processing, mathematics, functional-style programming, file/directory access, data persistence, data compression/archiving, cryptography, operating-system services, concurrent programming, interprocess communication, networking protocols, JSON/XML/other Internet data formats, multimedia, internationalization, GUI, debugging, profiling and more. The following table lists some of the Python Standard Library modules that we use in examples.

--	--

collections—Additional data structures beyond lists, tuples, dictionaries and sets.

csv—Processing comma-separated value files.

datetime, time—Date and time manipulations.

decimal—Fixed-point and floating-point arithmetic, including monetary calculations.

doctest—Simple unit testing via validation tests and expected results embedded in docstrings.

json—JavaScript Object Notation (JSON) processing for use with web services and NoSQL document databases.

math—Common math constants and operations.

os—Interacting with the operating system.

queue—First-in, first-out data structure.

random—Pseudorandom numbers.

re—Regular expressions for pattern matching.

sqlite3—SQLite relational database access.

statistics—Mathematical statistics functions like `mean`, `median`, `mode` and `variance`.

string—String processing.

sys—Command-line argument processing; standard input, standard output and standard error streams.

timeit—Performance analysis.

1.4.2 Data-Science Libraries

Python has an enormous and rapidly growing community of open-source developers in many fields. One of the biggest reasons for Python's popularity is the extraordinary range of open-source libraries developed by its open-source community. One of our goals is to create examples and implementation case studies that give you an engaging, challenging and entertaining introduction to Python programming, while also involving you in hands-on data science, key data-science libraries and more. You'll be amazed at the substantial tasks you can accomplish in just a few lines of code. The following table lists various popular data-science libraries. You'll use many of these as you work through our data-science examples. For visualization, we'll use Matplotlib, Seaborn and Folium, but there are many more. For a nice summary of Python visualization libraries see <http://pyviz.org/>.

Scientific Computing and Statistics

NumPy (Numerical Python)—Python does not have a built-in array data structure. It uses lists, which are convenient but relatively slow. NumPy provides the high-performance `ndarray` data structure to represent lists and matrices, and it also provides routines for processing such data structures.

SciPy (Scientific Python)—Built on NumPy, SciPy adds routines for scientific processing, such as integrals, differential equations, additional matrix processing and more. `scipy.org` controls SciPy and NumPy.

StatsModels—Provides support for estimations of statistical models, statistical tests and statistical data exploration.

Data Manipulation and Analysis

Pandas—An extremely popular library for data manipulations. Pandas makes abundant use of NumPy's `ndarray`. Its two key data structures are `Series` (one dimensional) and `DataFrames` (two dimensional).

Visualization

Matplotlib—A highly customizable visualization and plotting library. Supported plots include regular, scatter, bar, contour, pie, quiver, grid, polar axis, 3D and text.

Seaborn—A higher-level visualization library built on Matplotlib. Seaborn adds a nicer look-and-feel, additional visualizations and enables you to create visualizations with less code.

Machine Learning, Deep Learning and Reinforcement Learning

scikit-learn—Top machine-learning library. Machine learning is a subset of AI. Deep learning is a subset of machine learning that focuses on neural networks.

Keras—One of the easiest to use deep learning libraries. Keras runs on top of

Keras—One of the easiest to use deep learning libraries. Keras runs on top of TensorFlow (Google), CNTK (Microsoft's cognitive toolkit for deep learning) or Theano (Université de Montréal).

TensorFlow—From Google, this is the most widely used deep learning library. TensorFlow works with GPUs (graphics processing units) or Google's custom TPUs (Tensor processing units) for performance. TensorFlow is important in AI and big data analytics—where processing demands are huge. You'll use the version of Keras that's built into TensorFlow.

OpenAI Gym—A library and environment for developing, testing and comparing reinforcement-learning algorithms.

Natural Language Processing (NLP)

NLTK (Natural Language Toolkit)—Used for natural language processing (NLP) tasks.

TextBlob—An object-oriented NLP text-processing library built on the NLTK and pattern NLP libraries. TextBlob simplifies many NLP tasks.

Gensim—Similar to NLTK. Commonly used to build an index for a collection of documents, then determine how similar another document is to each of those in the index.

1.5 TEST-DRIVES: USING IPYTHON AND JUPYTER NOTEBOOKS

In this section, you'll test-drive the IPython interpreter⁹ in two modes:

⁹Before reading this section, follow the instructions in the Before You Begin section to install the Anaconda- Python distribution, which contains the IPython interpreter.

- In **interactive mode**, you'll enter small bits of Python code called **snippets** and immediately see their results.
- In **script mode**, you'll execute code loaded from a file that has the `.py` extension (short

for Python). Such files are called **scripts** or **programs**, and they're generally longer than the code snippets you'll use in interactive mode.

Then, you'll learn how to use the browser-based environment known as the Jupyter Notebook for writing and executing Python code.⁹

⁹Jupyter supports many programming languages by installing their "kernels." For more information see <https://github.com/jupyter/jupyter/wiki/Jupyter-kernels>.

1.5.1 Using IPython Interactive Mode as a Calculator

Let's use IPython interactive mode to evaluate simple arithmetic expressions.

Entering IPython in Interactive Mode

First, open a command-line window on your system:

- On macOS, open a **Terminal** from the **Applications** folder's **Utilities** subfolder.
- On Windows, open the **Anaconda Command Prompt** from the start menu.
- On Linux, open your system's **Terminal** or shell (this varies by Linux distribution).

In the command-line window, type `ipython`, then press *Enter* (or *Return*). You'll see text like the following, this varies by platform and by IPython version:

[lick here to view code image](#)

```
Python 3.7.0 | packaged by conda-forge | (default, Jan 20 2019, 17:24:52)
Type 'copyright', 'credits' or 'license' for more information
IPython 6.5.0 -- An enhanced Interactive Python. Type '?'
for help.

In [1]:
```

The text "`In [1]:`" is a *prompt*, indicating that IPython is waiting for your input. You can type `?` for help or begin entering snippets, as you'll do momentarily.

Evaluating Expressions

In interactive mode, you can evaluate expressions:

```
In [1]: 45 + 72
Out[1]: 117

In [2]:
```

After you type `45 + 72` and press *Enter*, IPython *reads* the snippet, *evaluates* it and *prints* its result in `Out[1]`.¹ Then IPython displays the `In [2]` prompt to show that it's waiting for you to enter your second snippet. For each new snippet, IPython adds 1 to the number in the square brackets. Each `In [1]` prompt in the book indicates that we've started a new interactive session. We generally do that for each new section of a chapter.

¹In the next chapter, you'll see that there are some cases in which `Out[]` is not displayed.

Let's evaluate a more complex expression:

[click here to view code image](#)

```
In [2]: 5 * (12.7 - 4) / 2
Out[2]: 21.75
```

Python uses the asterisk (*) for multiplication and the forward slash (/) for division. As in mathematics, parentheses force the evaluation order, so the parenthesized expression `(12.7 - 4)` evaluates first, giving `8.7`. Next, `5 * 8.7` evaluates giving `43.5`. Then, `43.5 / 2` evaluates, giving the result `21.75`, which IPython displays in `Out[2]`. Whole numbers, like 5, 4 and 2, are called **integers**. Numbers with decimal points, like 12.7, 43.5 and 21.75, are called **floating-point numbers**.

Exiting Interactive Mode

To leave interactive mode, you can:

- Type the `exit` command at the current `In []` prompt and press *Enter* to exit immediately.
- Type the key sequence `<Ctrl> + d` (or `<control> + d`). This displays the prompt "Do you really want to exit ([y]/n)?". The square brackets around `y` indicate that it's the default response—pressing *Enter* submits the default response and exits.
- Type `<Ctrl> + d` (or `<control> + d`) twice (macOS and Linux only).

1.5.2 Executing a Python Program Using the IPython Interpreter

In this section, you'll execute a script named `RollDieDynamic.py` that you'll write in [chapter 6](#). The **.py extension** indicates that the file contains Python source code. The script `RollDieDynamic.py` simulates rolling a six-sided die. It presents a colorful animated visualization that dynamically graphs the frequencies of each die face.

Changing to This Chapter's Examples Folder

You'll find the script in the book's `ch01` source-code folder. In the *Before You Begin* section you extracted the `examples` folder to your user account's `Documents` folder. Each chapter

has a folder containing that chapter's source code. The folder is named `ch##`, where `##` is a two-digit chapter number from 01 to 17. First, open your system's command-line window. Next, use the `cd` ("change directory") command to change to the `ch01` folder:

- On macOS/Linux, type `cd ~/Documents/examples/ch01`, then press *Enter*.
- On Windows, type `cd C:\Users\YourAccount\Documents\examples\ch01`, then press *Enter*.

Executing the Script

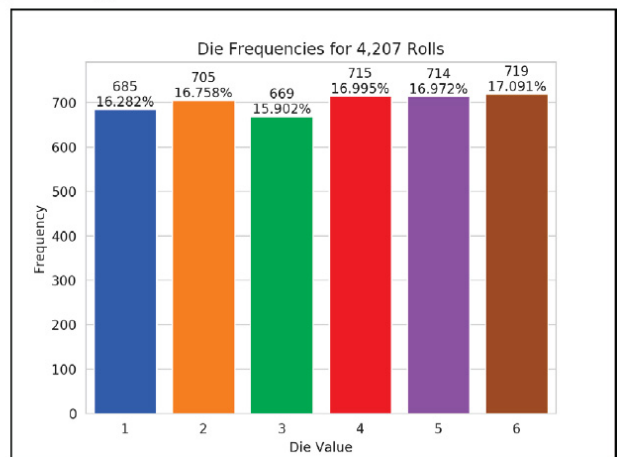
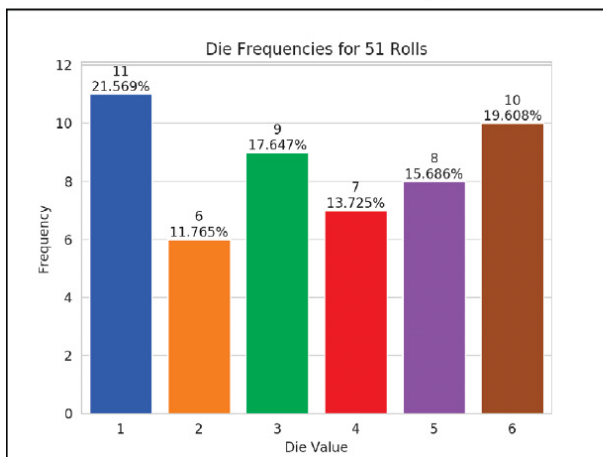
To execute the script, type the following command at the command line, then press *Enter*:

```
ipython RollDieDynamic.py 6000 1
```

The script displays a window, showing the visualization. The numbers `6000` and `1` tell this script the number of times to roll dice and how many dice to roll each time. In this case, we'll update the chart `6000` times for `1` die at a time.

For a six-sided die, the values 1 through 6 should each occur with "equal likelihood"—the probability of each is $1/6^{\text{th}}$ or about 16.667%. If we roll a die 6000 times, we'd expect about 1000 of each face. Like coin tossing, die rolling is *random*, so there could be some faces with fewer than 1000, some with 1000 and some with more than 1000. We took the screen captures below during the script's execution. This script uses randomly generated die values, so your results will differ. Experiment with the script by changing the value `1` to `100`, `1000` and `10000`. Notice that as the number of die rolls gets larger, the frequencies zero in on 16.667%. This is a phenomenon of the "Law of Large Numbers."

Roll the dice 6000 times and roll 1 die each time:
`ipython RollDieDynamic.py 6000 1`



Creating Scripts

Typically, you create your Python source code in an editor that enables you to type text. Using the editor, you type a program, make any necessary corrections and save it to your computer.

Integrated development environments (IDEs) provide tools that support the entire

software-development process, such as editors, debuggers for locating logic errors that cause programs to execute incorrectly and more. Some popular Python IDEs include Spyder (which comes with Anaconda), PyCharm and Visual Studio Code.

Problems That May Occur at Execution Time

Programs often do not work on the first try. For example, an executing program might try to divide by zero (an illegal operation in Python). This would cause the program to display an error message. If this occurred in a script, you'd return to the editor, make the necessary corrections and re-execute the script to determine whether the corrections fixed the problem(s).

Errors such as division by zero occur as a program runs, so they're called **runtime errors** or **execution-time errors**. **Fatal runtime errors** cause programs to terminate immediately without having successfully performed their jobs. **Non-fatal runtime errors** allow programs to run to completion, often producing incorrect results.

1.5.3 Writing and Executing Code in a Jupyter Notebook

The Anaconda Python Distribution that you installed in the Before You Begin section comes with the **Jupyter Notebook**—an interactive, browser-based environment in which you can write and execute code and intermix the code with text, images and video. Jupyter Notebooks are broadly used in the data-science community in particular and the broader scientific community in general. They're the preferred means of doing Python-based data analytics studies and *reproducibly* communicating their results. The Jupyter Notebook environment supports a growing number of programming languages.

For your convenience, all of the book's source code also is provided in Jupyter Notebooks that you can simply load and execute. In this section, you'll use the **JupyterLab** interface, which enables you to manage your notebook files and other files that your notebooks use (like images and videos). As you'll see, JupyterLab also makes it convenient to write code, execute it, see the results, modify the code and execute it again.

You'll see that coding in a Jupyter Notebook is similar to working with IPython—in fact, Jupyter Notebooks use IPython by default. In this section, you'll create a notebook, add the code from [Section 1.5.1](#) to it and execute that code.

Opening JupyterLab in Your Browser

To open JupyterLab, change to the `ch01` examples folder in your Terminal, shell or Anaconda Command Prompt (as in [Section 1.5.2](#)), type the following command, then press *Enter* (or *Return*):

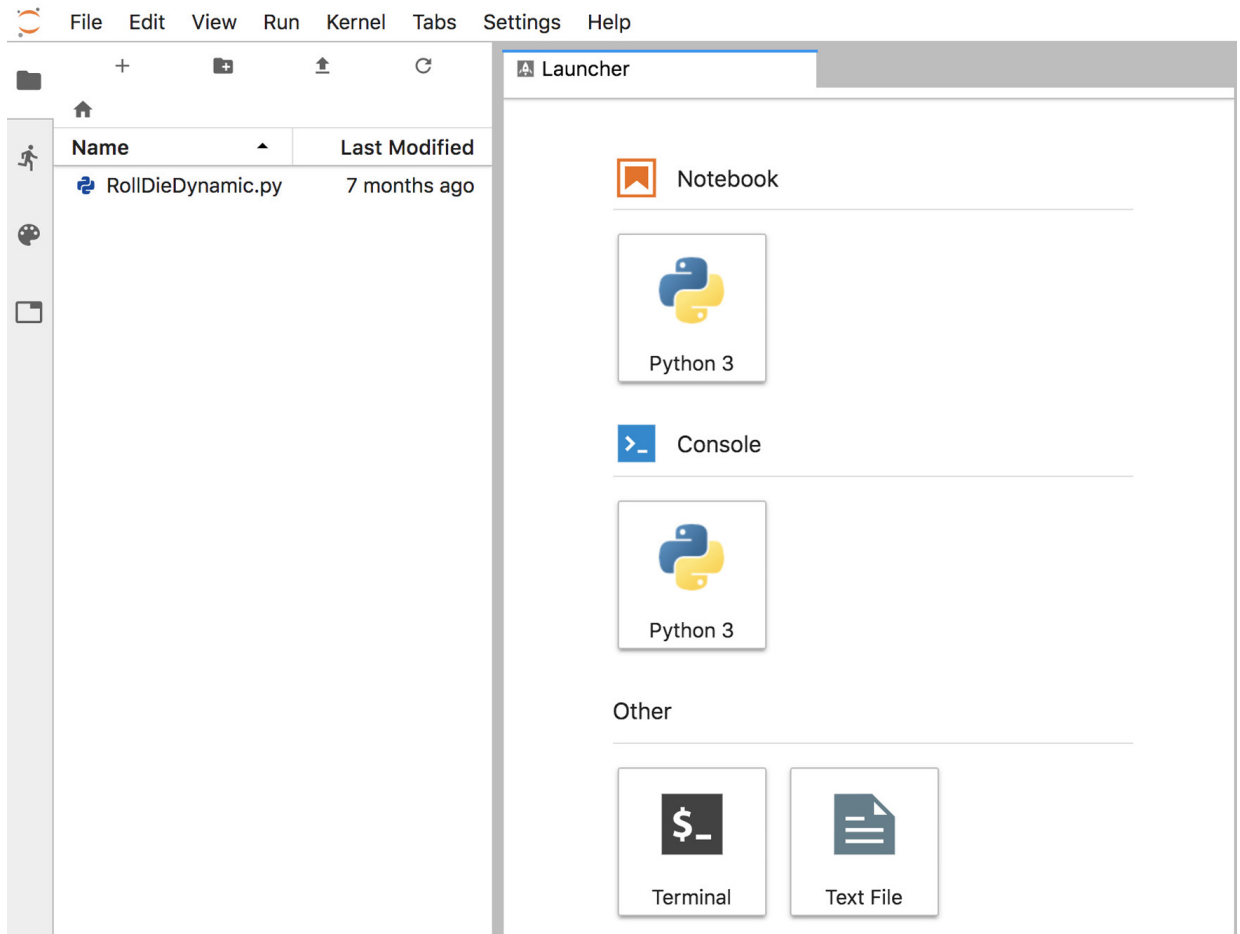
```
jupyter lab
```

This executes the Jupyter Notebook server on your computer and opens JupyterLab in your

default web browser, showing the `ch01` folder's contents in the **File Browser** tab



at the left side of the JupyterLab interface:



The Jupyter Notebook server enables you to load and run Jupyter Notebooks in your web browser. From the JupyterLab **Files** tab, you can double-click files to open them in the right side of the window where the **Launcher** tab is currently displayed. Each file you open appears as a separate tab in this part of the window. If you accidentally close your browser, you can reopen JupyterLab by entering the following address in your web browser

```
http://localhost:8888/lab
```

Creating a New Jupyter Notebook

In the **Launcher** tab under **Notebook**, click the **Python 3** button to create a new Jupyter Notebook named `Untitled.ipynb` in which you can enter and execute Python 3 code. The file extension `.ipynb` is short for IPython Notebook—the original name of the Jupyter Notebook.

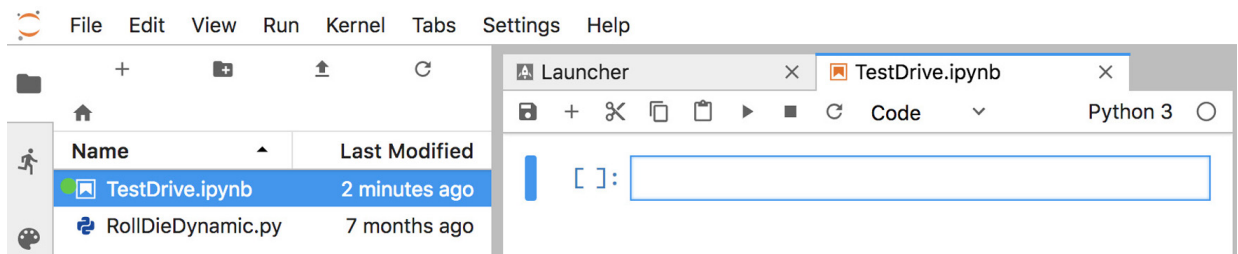
Renaming the Notebook

Rename `Untitled.ipynb` as `TestDrive.ipynb`:

1. Right-click the `Untitled.ipynb` tab and select **Rename Notebook**.

2. Change the name to `TestDrive.ipynb` and click **RENAME**.

The top of JupyterLab should now appear as follows:

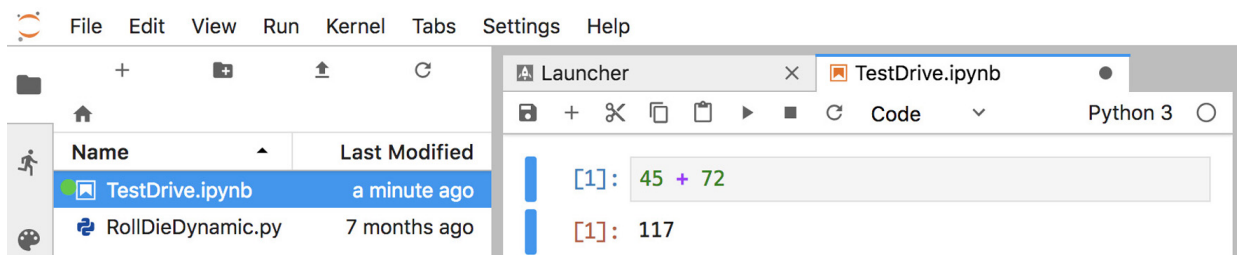


Evaluating an Expression

The unit of work in a notebook is a **cell** in which you can enter code snippets. By default, a new notebook contains one cell—the rectangle in the `TestDrive.ipynb` notebook—but you can add more. To the cell's left, the notation `[ ] :` is where the Jupyter Notebook will display the cell's snippet number *after* you execute the cell. Click in the cell, then type the expression

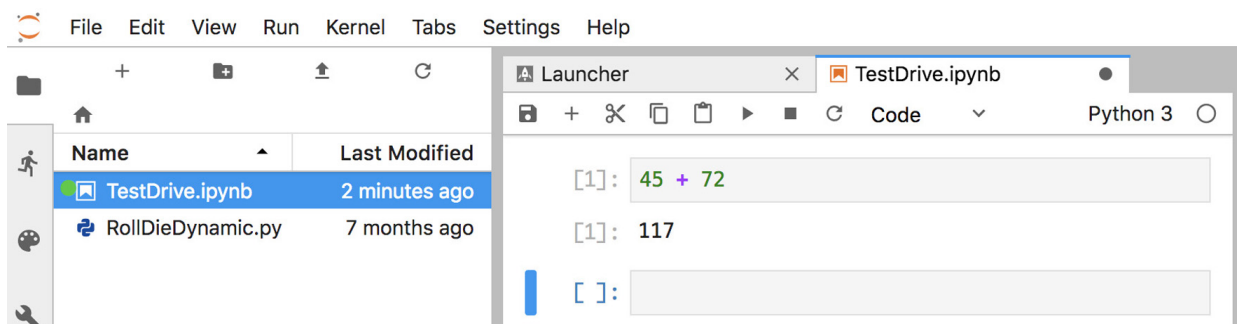
```
45 + 72
```

To execute the current cell's code, type *Ctrl + Enter* (or *control + Enter*). JupyterLab executes the code in IPython, then displays the results below the cell:



Adding and Executing Another Cell

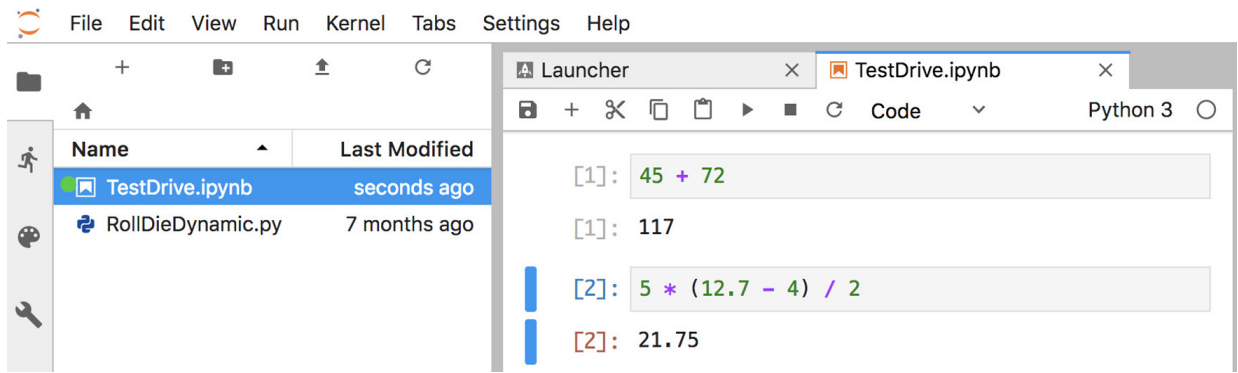
Let's evaluate a more complex expression. First, click the **+** button in the toolbar above the notebook's first cell—this adds a new cell below the current one:



Click in the new cell, then type the expression

```
5 * (12.7 - 4) / 2
```

and execute the cell by typing *Ctrl + Enter* (or *control + Enter*):



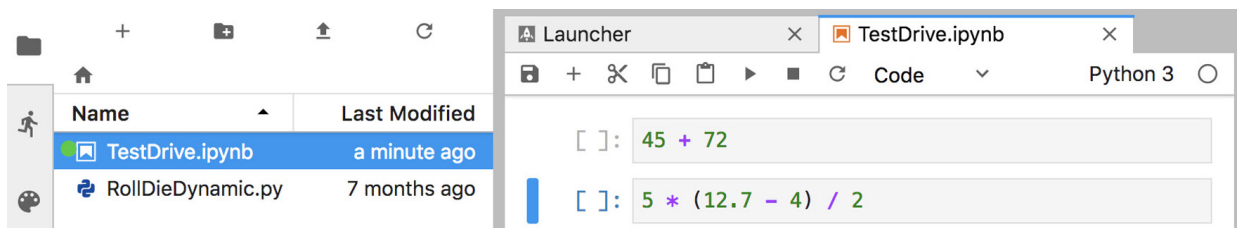
Saving the Notebook

If your notebook has unsaved changes, the **X** in the notebook's tab will change to **.** To save the notebook, select the **File** menu in JupyterLab (not at the top of your browser's window), then select **Save Notebook**.

Notebooks Provided with Each Chapter's Examples

For your convenience, each chapter's examples also are provided as ready-to-execute notebooks without their outputs. This enables you to work through them snippet-by-snippet and see the outputs appear as you execute each snippet.

So that we can show you how to load an existing notebook and execute its cells, let's reset the `TestDrive.ipynb` notebook to remove its output and snippet numbers. This will return it to a state like the notebooks we provide for the subsequent chapters' examples. From the **Kernel** menu select **Restart Kernel and Clear All Outputs...**, then click the **RESTART** button. The preceding command also is helpful whenever you wish to re-execute a notebook's snippets. The notebook should now appear as follows:



From the **File** menu, select **Save Notebook**, then click the `TestDrive.ipynb` tab's **X** button to close the notebook.

Opening and Executing an Existing Notebook

When you launch JupyterLab from a given chapter's examples folder, you'll be able to open notebooks from that folder or any of its subfolders. Once you locate a specific notebook, double-click it to open it. Open the `TestDrive.ipynb` notebook again now. Once a notebook is open, you can execute each cell individually, as you did earlier in this section, or you can execute the entire notebook at once. To do so, from the **Run** menu select **Run All**

Cells. The notebook will execute the cells in order, displaying each cell's output below that cell.

Closing JupyterLab

When you're done with JupyterLab, you can close its browser tab, then in the Terminal, shell or Anaconda Command Prompt from which you launched JupyterLab, type *Ctrl + c* (or *control + c*) twice.

JupyterLab Tips

While working in JupyterLab, you might find these tips helpful:

- If you need to enter and execute many snippets, you can execute the current cell *and* add a new one below it by typing *Shift + Enter*, rather than *Ctrl + Enter* (or *control + Enter*).
- As you get into the later chapters, some of the snippets you'll enter in Jupyter Notebooks will contain many lines of code. To display line numbers within each cell, select **Show line numbers** from JupyterLab's **View** menu.

More Information on Working with JupyterLab

JupyterLab has many more features that you'll find helpful. We recommend that you read the Jupyter team's introduction to JupyterLab at:

```
https://jupyterlab.readthedocs.io/en/stable/index.html
```

For a quick overview, click **Overview** under **GETTING STARTED**. Also, under **USER GUIDE** read the introductions to **The JupyterLab Interface**, **Working with Files**, **Text Editor** and **Notebooks** for many additional features.

1.6 THE CLOUD AND THE INTERNET OF THINGS

1.6.1 The Cloud

More and more computing today is done “in the cloud”—that is, distributed across the Internet worldwide. Many apps you use daily are dependent on **cloud-based services** that use massive clusters of computing resources (computers, processors, memory, disk drives, etc.) and databases that communicate over the Internet with each other and the apps you use. A service that provides access to itself over the Internet is known as a **web service**. As you'll see, using cloud-based services in Python often is as simple as creating a software object and interacting with it. That object then uses web services that connect to the cloud on your behalf.

Throughout the [chapters 11– 6](#) examples, you'll work with many cloud-based services:

- In chapters 12 and 6, you'll use Twitter's web services (via the Python library Tweepy) to get information about specific Twitter users, search for tweets from the last seven days and receive streams of tweets as they occur—that is, in real time.
- In chapters 11 and 2, you'll use the Python library TextBlob to translate text between languages. Behind the scenes, TextBlob uses the Google Translate web service to perform those translations.
- In chapter 13, you'll use the IBM Watson's Text to Speech, Speech to Text and Translate services. You'll implement a traveler's assistant translation app that enables you to speak a question in English, transcribes the speech to text, translates the text to Spanish and speaks the Spanish text. The app then allows you to speak a Spanish response (in case you don't speak Spanish, we provide an audio file you can use), transcribes the speech to text, translates the text to English and speaks the English response. Via IBM Watson demos, you'll also experiment with many other Watson cloud-based services in chapter 13.
- In chapter 16, you'll work with Microsoft Azure's HDInsight service and other Azure web services as you implement big-data applications using Apache Hadoop and Spark. Azure is Microsoft's set of cloud-based services.
- In chapter 16, you'll use the Dweet.io web service to simulate an Internet-connected thermostat that publishes temperature readings online. You'll also use a web-based service to create a “dashboard” that visualizes the temperature readings over time and warns you if the temperature gets too low or too high.
- In chapter 16, you'll use a web-based dashboard to visualize a simulated stream of live sensor data from the PubNub web service. You'll also create a Python app that visualizes a PubNub simulated stream of live stock-price changes.

In most cases, you'll create Python objects that interact with web services on your behalf, hiding the details of how to access these services over the Internet.

Mashups

The applications-development methodology of *mashups* enables you to rapidly develop powerful software applications by combining (often free) complementary web services and other forms of information feeds—as you'll do in our IBM Watson traveler's assistant translation app. One of the first mashups combined the real-estate listings provided by <http://www.craigslist.org> with the mapping capabilities of Google Maps to offer maps that showed the locations of homes for sale or rent in a given area.

ProgrammableWeb (<http://www.programmableweb.com/>) provides a directory of over 20,750 web services and almost 8,000 mashups. They also provide how-to guides and sample code for working with web services and creating your own mashups. According to their website, some of the most widely used web services are Facebook, Google Maps, Twitter and

ouTube.

1.6.2 Internet of Things

The Internet is no longer just a network of *computers*—it's an **Internet of Things (IoT)**. A *thing* is any object with an IP address and the ability to send, and in some cases receive, data automatically over the Internet. Such *things* include:

- a car with a transponder for paying tolls,
- monitors for parking-space availability in a garage,
- a heart monitor implanted in a human,
- water quality monitors,
- a smart meter that reports energy usage,
- radiation detectors,
- item trackers in a warehouse,
- mobile apps that can track your movement and location,
- smart thermostats that adjust room temperatures based on weather forecasts and activity in the home, and
- intelligent home appliances.

According to `statista.com`, there are already over 23 billion IoT devices in use today, and there could be over 75 billion IoT devices in 2025.²

² <https://www.statista.com/statistics/471264/iot-number-of-connected-devices-worldwide/>.

1.7 HOW BIG IS BIG DATA?

For computer scientists and data scientists, data is now as important as writing programs. According to IBM, approximately 2.5 quintillion bytes (2.5 *exabytes*) of data are created daily,³ and 90% of the world's data was created in the last two years.⁴ According to IDC, the global data supply will reach 175 *zettabytes* (equal to 175 trillion gigabytes or 175 billion terabytes) annually by 2025.⁵ Consider the following examples of various popular data measures.

³ <https://www.ibm.com/blogs/watson/2016/06/welcome-to-the-world-of-a->
/.

⁴ <https://public.dhe.ibm.com/common/ssi/ecm/wr/en/wr112345usen/watson-customer-engagement--watson-marketing-wr-other-papers-and-reports-r112345usen-20170719.pdf>.

⁵ <https://www.networkworld.com/article/3325397/storage/idc-expect-75-zettabytes-of-data-worldwide-by-2025.html>.

Megabytes (MB)

One megabyte is about one million (actually 2^{20}) bytes. Many of the files we use on a daily basis require one or more MBs of storage. Some examples include:

- MP3 audio files—High-quality MP3s range from 1 to 2.4 MB per minute.⁶

⁶ <https://www.audiomountain.com/tech/audio-file-size.html>.

- Photos—JPEG format photos taken on a digital camera can require about 8 to 10 MB per photo.
- Video—Smartphone cameras can record video at various resolutions. Each minute of video can require many megabytes of storage. For example, on one of our iPhones, the **Camera** settings app reports that 1080p video at 30 frames-per-second (FPS) requires 130 MB/minute and 4K video at 30 FPS requires 350 MB/minute.

Gigabytes (GB)

One gigabyte is about 1000 megabytes (actually 2^{30} bytes). A dual-layer DVD can store up to 8.5 GB⁷, which translates to:

⁷ <https://en.wikipedia.org/wiki/DVD>.

- as much as 141 hours of MP3 audio,
- approximately 1000 photos from a 16-megapixel camera,
- approximately 7.7 minutes of 1080p video at 30 FPS, or
- approximately 2.85 minutes of 4K video at 30 FPS.

The current highest-capacity Ultra HD Blu-ray discs can store up to 100 GB of video.⁸ Streaming a 4K movie can use between 7 and 10 GB per hour (highly compressed).

⁸ https://en.wikipedia.org/wiki/Ultra_HD_Blu-ray.

Terabytes (TB)

One terabyte is about 1000 gigabytes (actually 2^{40} bytes). Recent disk drives for desktop

computers come in sizes up to 15 TB,⁹ which is equivalent to:

⁹ <https://www.zdnet.com/article/worlds-biggest-hard-drive-meet-eastern-digital-15tb-monster/>.

- approximately 28 years of MP3 audio,
- approximately 1.68 million photos from a 16-megapixel camera,
- approximately 226 hours of 1080p video at 30 FPS and
- approximately 84 hours of 4K video at 30 FPS.

Nimbus Data now has the largest solid-state drive (SSD) at 100 TB, which can store 6.67 times the 15-TB examples of audio, photos and video listed above.⁹

⁹ <https://www.cinema5d.com/nimbus-data-100tb-ssd-worlds-largest-ssd/>.

Petabytes, Exabytes and Zettabytes

There are nearly four billion people online creating about 2.5 quintillion bytes of data each day¹—that's 2500 petabytes (each petabyte is about 1000 terabytes) or 2.5 exabytes (each exabyte is about 1000 petabytes). According to a March 2016 *AnalyticsWeek* article, within five years there will be over 50 billion devices connected to the Internet (most of them through the Internet of Things, which we discuss in sections 1.6.2 and 6.8) and by 2020 we'll be producing 1.7 megabytes of new data every second *for every person on the planet*.² At today's numbers (approximately 7.7 billion people³), that's about

¹ <https://public.dhe.ibm.com/common/ssi/ecm/wr/en/wr112345usen/watson-customer-engagement--watson-marketing-wr-other-papers-and-reports-r112345usen-20170719.pdf>.

² <https://analyticsweek.com/content/big-data-facts/>.

³ https://en.wikipedia.org/wiki/World_population.

- 13 petabytes of new data per second,
- 780 petabytes per minute,
- 46,800 petabytes (46.8 exabytes) per hour and
- 1,123 exabytes per day—that's 1.123 zettabytes (ZB) per day (each zettabyte is about 1000 exabytes).

That's the equivalent of over 5.5 million hours (over 600 years) of 4K video every day or

pproximately 116 billion photos every day!

Additional Big-Data Stats

For an entertaining real-time sense of big data, check out

<https://www.internetlivestats.com>, with various statistics, including the numbers so far today of

- Google searches.
- Tweets.
- Videos viewed on YouTube.
- Photos uploaded on Instagram.

You can click each statistic to drill down for more information. For instance, they say over 250 *billion* tweets were sent in 2018.

Some other interesting big-data facts:

- Every hour, YouTube users upload 24,000 hours of video, and almost 1 billion hours of video are watched on YouTube every day.⁴

⁴ <https://www.brandwatch.com/blog/youtube-stats/>.

- Every second, there are 51,773 GBs (or 51.773 TBs) of Internet traffic, 7894 tweets sent, 64,332 Google searches and 72,029 YouTube videos viewed.⁵

⁵ <http://www.internetlivestats.com/one-second>.

- On Facebook each day there are 800 million “likes,”⁶ 60 million emojis are sent,⁷ and there are over two billion searches of the more than 2.5 trillion Facebook posts since the site’s inception.⁸

⁶ <https://newsroom.fb.com/news/2017/06/two-billion-people-coming-together-on-facebook>.

⁷ <https://mashable.com/2017/07/17/facebook-world-emoji-day/>.

⁸ <https://techcrunch.com/2016/07/27/facebook-will-make-you-talk/>.

- In June 2017, Will Marshall, CEO of Planet, said the company has 142 satellites that image the whole planet’s land mass once per day. They add one million images and seven TBs of new data each day. Together with their partners, they’re using machine learning on that data to improve crop yields, see how many ships are in a given port and track

eforestation. With respect to Amazon deforestation, he said: “Used to be we’d wake up after a few years and there’s a big hole in the Amazon. Now we can literally count every tree on the planet every day.”⁹

⁹ <https://www.bloomberg.com/news/videos/2017-06-30/learning-from-planet-s-shoe-boxed-sized-satellites-video>, June 30, 2017.

Domo, Inc. has a nice infographic called “Data Never Sleeps 6.0” showing how much data is generated *every minute*, including:⁹

⁹ <https://www.domo.com/learn/data-never-sleeps-6>.

- 473,400 tweets sent.
- 2,083,333 Snapchat photos shared.
- 97,222 hours of Netflix video viewed.
- 12,986,111 million text messages sent.
- 49,380 Instagram posts.
- 176,220 Skype calls.
- 750,000 Spotify songs streamed.
- 3,877,140 Google searches.
- 4,333,560 YouTube videos watched.

Computing Power Over the Years

Data is getting more massive and so is the computing power for processing it. The performance of today’s processors is often measured in terms of **FLOPS (floating-point operations per second)**. In the early to mid-1990s, the fastest supercomputer speeds were measured in gigaflops (10^9 FLOPS). By the late 1990s, Intel produced the first teraflop (10^{12} FLOPS) supercomputers. In the early-to-mid 2000s, speeds reached hundreds of teraflops, then in 2008, IBM released the first petaflop (10^{15} FLOPS) supercomputer. Currently, the fastest supercomputer—the IBM Summit, located at the Department of Energy’s (DOE) Oak Ridge National Laboratory (ORNL)—is capable of 122.3 peta-flops.¹

¹ <https://en.wikipedia.org/wiki/FLOPS>.

Distributed computing can link thousands of personal computers via the Internet to produce even more FLOPS. In late 2016, the Folding@home network—a distributed network in which people volunteer their personal computers’ resources for use in disease research and drug